

Resumen del algoritmo TF-IDF

TF-IDF (Term Frequency – Inverse Document Frequency) es un método para evaluar la importancia de una palabra dentro de un documento *en relación* al resto de documentos de un corpus.

Se usa mucho en **procesamiento de lenguaje natural, búsqueda de información y machine learning**.

1. ¿Qué mide TF-IDF?

Mide dos cosas combinadas:

TF – Frecuencia del término

Indica *cuántas veces aparece una palabra en un documento*.

- Una palabra que aparece muchas veces es más importante dentro de ese documento.
- Se suele normalizar para evitar que documentos largos dominen.

Ejemplo de fórmula:

$$TF(t, d) = \frac{\text{número de veces que aparece } t \text{ en } d}{\text{total de palabras en } d}$$

IDF – Frecuencia inversa de documentos

Mide *lo común o rara* que es una palabra en el corpus.

- Las palabras **muy comunes** (como "el", "de", "y") tienen un IDF **bajo**.
- Las palabras **raras** tienen un IDF **alto** → son más informativas.

Fórmula típica:

$$IDF(t) = \log \left(\frac{N}{1 + DF(t)} \right)$$

donde:

- N es el número total de documentos
 - $DF(t)$ es la cantidad de documentos donde aparece el término t
-

2. Cálculo de TF-IDF

El valor final se obtiene multiplicando:

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

Interpretación:

- Alto → la palabra es frecuente en el documento y rara en el corpus.
 - Bajo → o es poco frecuente, o aparece en muchos documentos.
-

3. ¿Para qué sirve?

- **Detección de palabras relevantes**

Permite identificar los términos más importantes de un texto.

- **Vectorización para machine learning**

Convierte documentos a vectores numéricos para clasificadores, clustering, etc.

- **Motores de búsqueda**

Ayuda a comparar documentos con consultas y ordenar resultados por relevancia.

4. Ventajas y limitaciones

Ventajas

- Simple y efectivo.
- Muy útil en tareas clásicas de NLP.
- No requiere modelos entrenados.

Limitaciones

- No captura significado (solo frecuencia).

- No entiende contexto ni sinónimos.
- No funciona bien con documentos extremadamente cortos o largos.

Ejemplo sencillo de TF-IDF con 3 documentos

Supongamos estos documentos:

D1: "gato blanco"

D2: "gato negro"

D3: "perro blanco"

Vocabulario

Palabras únicas:

Término
gato
blanco
negro
perro

Cálculo del TF (Term Frequency)

TF = nº apariciones / nº total palabras del documento

Documento 1: "gato blanco"

2 palabras → cada palabra aparece 1 vez

- $TF(\text{gato}, D1) = 1/2$
- $TF(\text{blanco}, D1) = 1/2$
- $TF(\text{negro}, D1) = 0$
- $TF(\text{perro}, D1) = 0$

Documento 2: "gato negro"

2 palabras

- $TF(\text{gato}, D2) = 1/2$
- $TF(\text{negro}, D2) = 1/2$
- $TF(\text{blanco}, D2) = 0$
- $TF(\text{perro}, D2) = 0$

Documento 3: "perro blanco"

2 palabras

- $TF(\text{perro}, D3) = 1/2$
- $TF(\text{blanco}, D3) = 1/2$
- $TF(\text{gato}, D3) = 0$
- $TF(\text{negro}, D3) = 0$

Cálculo del DF (Document Frequency)

- Número de documentos donde aparece cada término:

Término	DF
gato	2
blanco	2
negro	1
perro	1

Cálculo del IDF

Usamos:

$$IDF(t) = \log \left(\frac{N}{DF(t)} \right)$$

Con $N = 3$ documentos.

- $IDF(\text{gato}) = \log(3/2)$
- $IDF(\text{blanco}) = \log(3/2)$
- $IDF(\text{negro}) = \log(3/1)$
- $IDF(\text{perro}) = \log(3/1)$

Aproximando:

Término	IDF
gato	0.176
blanco	0.176
negro	1.099
perro	1.099

Cálculo del TF-IDF = TF × IDF

Documento 1: "gato blanco"

Término	TF	IDF	TF-IDF
gato	0.5	0.176	0.088
blanco	0.5	0.176	0.088
negro	0	1.099	0
perro	0	1.099	0

Documento 2: "gato negro"

Término	TF	IDF	TF-IDF
gato	0.5	0.176	0.088
negro	0.5	1.099	0.549
blanco	0	0.176	0
perro	0	1.099	0

Documento 3: "perro blanco"

Término	TF	IDF	TF-IDF
perro	0.5	1.099	0.549
blanco	0.5	0.176	0.088
gato	0	0.176	0
negro	0	1.099	0

Matriz TF-IDF final (documentos × términos)

Formato: filas = documentos, columnas = términos

Documento	gato	blanco	negro	perro
D1	0.088	0.088	0	0
D2	0.088	0	0.549	0
D3	0	0.088	0	

Código Python del ejemplo TF-IDF

```
import math
import pandas as pd

# =====
# 1. Documentos
# =====
docs = [
    "gato blanco",
    "gato negro",
    "perro blanco"
]

# =====
# 2. Tokenización
# =====
tokenized = [doc.split() for doc in docs]

# =====
# 3. Construcción del vocabulario
# =====
vocab = sorted({word for doc in tokenized for word in doc})

print("Vocabulario:", vocab)

# =====
# 4. Cálculo del TF
# =====
def compute_tf(doc_words, vocab):
    N = len(doc_words)
    return {term: doc_words.count(term) / N for term in vocab}

TF = [compute_tf(doc, vocab) for doc in tokenized]

# =====
# 5. Cálculo del DF
# =====
def compute_df(tokenized_docs, vocab):
    df = {}
    for term in vocab:
        df[term] = sum(term in doc for doc in tokenized_docs)
    return df

DF = compute_df(tokenized, vocab)

# =====
# 6. Cálculo del IDF
# =====
```

```

N_docs = len(docs)
IDF = {term: math.log(N_docs / DF[term]) for term in vocab}

print("\nIDF:", IDF)

# =====
# 7. Cálculo del TF-IDF por documento
# =====
TFIDF = []
for tf_doc in TF:
    tfidf_doc = {term: tf_doc[term] * IDF[term] for term in vocab}
    TFIDF.append(tfidf_doc)

# =====
# 8. Convertir a DataFrame
# =====
df_tfidf = pd.DataFrame(TFIDF, index=["D1", "D2", "D3"])

print("\nMatriz TF-IDF:\n")
print(df_tfidf)

```

Código Python sklearn TF-IDF

```

from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

# 1. Documentos
docs = [
    "gato blanco",
    "gato negro",
    "perro blanco"
]

# 2. Crear el vectorizador
vectorizer = TfidfVectorizer()

# 3. Ajustar y transformar
tfidf_matrix = vectorizer.fit_transform(docs)

# 4. Obtener vocabulario (columnas)
terms = vectorizer.get_feature_names() # <- la versión compatible

# 5. Convertir a DataFrame
df_tfidf = pd.DataFrame(tfidf_matrix.toarray(), columns=terms,
index=["D1", "D2", "D3"])

print(df_tfidf)

```

	blanco	gato	negro	perro
D1	0.707107	0.707107	0.000000	0.000000
D2	0.000000	0.605349	0.795961	0.000000
D3	0.605349	0.000000	0.000000	0.795961

Resumen de por qué los resultados NO coinciden

Razón	Fórmulas manuales	sklearn
TF	frecuencia relativa (0.5)	frecuencia absoluta (1)
IDF	$\log(N/DF)$	$\log((1+N)/(1+DF)) + 1$
Normalización	✗ No	✓ L2 automática
Corrección para DF=0	✗ No	✓ Sí

Por eso los números de sklearn siempre son distintos, incluso aunque el ranking de importancia siga siendo el mismo.

1. ¿Qué representan los valores TF-IDF?

Un valor TF-IDF alto significa:

- ✓ Esa palabra es **importante en ese documento**
- ✓ Y **no es común en el resto**

Por eso TF-IDF sirve para identificar **términos relevantes** en textos.

2. ¿Qué puedo hacer con los valores TF-IDF?

Aquí tienes los usos **más típicos y útiles**:

2.1. Encontrar las palabras más importantes de cada documento

Muy usado para:

- Resumir textos
- Etiquetado automático (tags)
- Entender de qué habla cada documento

Ejemplo:

```
import numpy as np

for i, row in df_tfidf.iterrows():
    top = row.sort_values(ascending=False)
```



```
print(f"\nDocumento {i}:")
print(top.head(3))
```

2.2. Convertir documentos en vectores numéricos para Machine Learning

TF-IDF convierte cada documento en un vector (una fila de la matriz).
Con eso puedes alimentar modelos como:

- **SVM**
- **Regresiones logísticas**
- **Árboles**
- **Redes neuronales**
- **K-means** (clustering)
- **KNN**
- Cualquier modelo de scikit-learn

Ejemplo de clasificación:

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(tfidf_matrix, etiquetas)
```

2.3. Búsqueda de documentos similares (search engine)

Con TF-IDF puedes comparar textos calculando similitudes:

- Similitud de coseno
- Distancias euclídeas
- Comparar consultas con documentos

Ejemplo:

```
from sklearn.metrics.pairwise import cosine_similarity

sim = cosine_similarity(tfidf_matrix)
print(sim)
```

Esto te da qué documentos son parecidos entre sí.

2.4. Filtrar palabras poco relevantes en análisis de texto

Por ejemplo, eliminar palabras cuyo TF-IDF sea:

- muy bajo → probablemente “ruido”
- muy alto → extremadamente específicas

2.5. Generar resúmenes automáticos de texto

Normalmente:

- Ordenas palabras por TF-IDF
- Seleccionas las que mejor explican el contenido

2.6. Clusterizar documentos para agruparlos

Con K-Means sobre la matriz TF-IDF, puedes agrupar textos parecidos:

```
from sklearn.cluster import KMeans  
  
kmeans = KMeans(n_clusters=2)  
kmeans.fit(tfidf_matrix)
```